The Inner Workings of Compilers

Jyotisko Sengupta

Introduction

Computers do not understand high level languages like C, Python, etc.

They only understand machine code (binary).

Compilers and interpreters bridge this gap by converting human readable code written in high-level-languages to machine code.

This presentation explores how high-level code is converted to machine executable instructions by understanding the inner workings of compilers.

Compiler vs Interpreter

Compiler

Compiler translates the entire code at once.

Fast.

Shows all errors before run.

Creates separate platform specific executable file.

Example: C, C++ using GCC

Interpreter

Interpreter translates and executes the code line by line.

Slow.

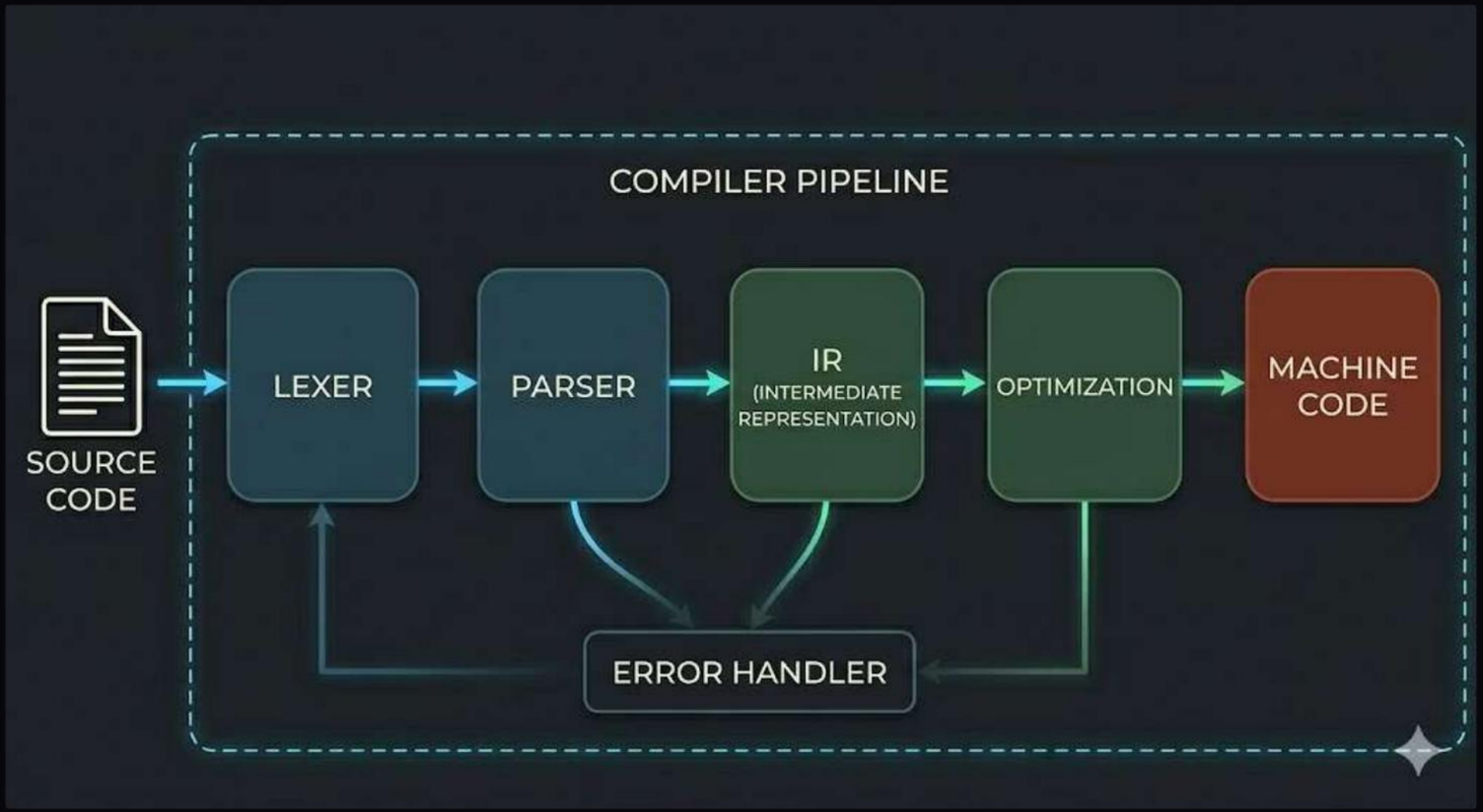
Stops at first error.

Does not create separate executable file, hence requires interpreter installed.

Example: Python using CPython

Compilers optimize before execution, interpreters execute directly

Compiler Pipeline



- Lexer: Breaks down the code into small pieces called tokens or meaningful units.
- Parser: Checks structure and builds meaning through Abstract Syntax Tree (AST).
- Intermediate Representation: Converts structured code into simplified, platform-independent form that can run on any CPU.
- Optimization: Removes unnecessary work and improves efficiency.
- Machine Code: Executed by the computer directly.

Compiler Pipeline (Example)

1. Source Code

```
x = 5 + 7 * 10
```

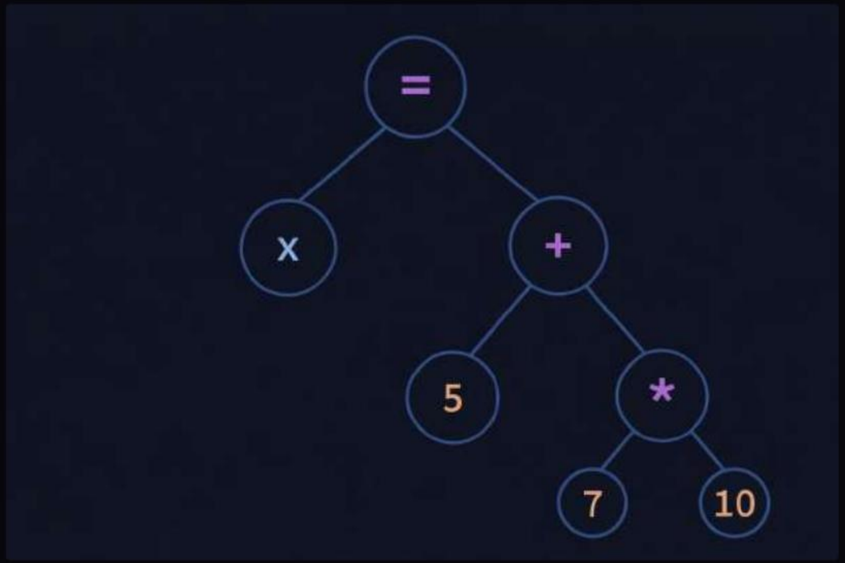
2. Lexer (Lexical Analysis)

Lexemes	Tokens
X	Identifier
=	Operator
5	Constant
+	Operator
7	Constant
*	Operator
10	Constant

3. Parser

x = 5 + **(7 * 10)**

4. AST (Abstract Syntax Tree)



5. IR (Intermediate Representation)

```
t1 = 7 * 10
t2 = 5 + t1
x = t2
```

6. Optimization → Machine Code

```
x = 75
```

Challenges: With Compilers

Platform Independence

Different systems use different architectures (x86 or ARM). AST is converted to Intermediate Representation (IR) which acts a machine independent layer.

Hardware Specific Limitations

Registers are CPU's working memory. Different CPUs have different number of registers and different memory hierarchy. Compiler needs to decide what to store in registers and what to store in memory.

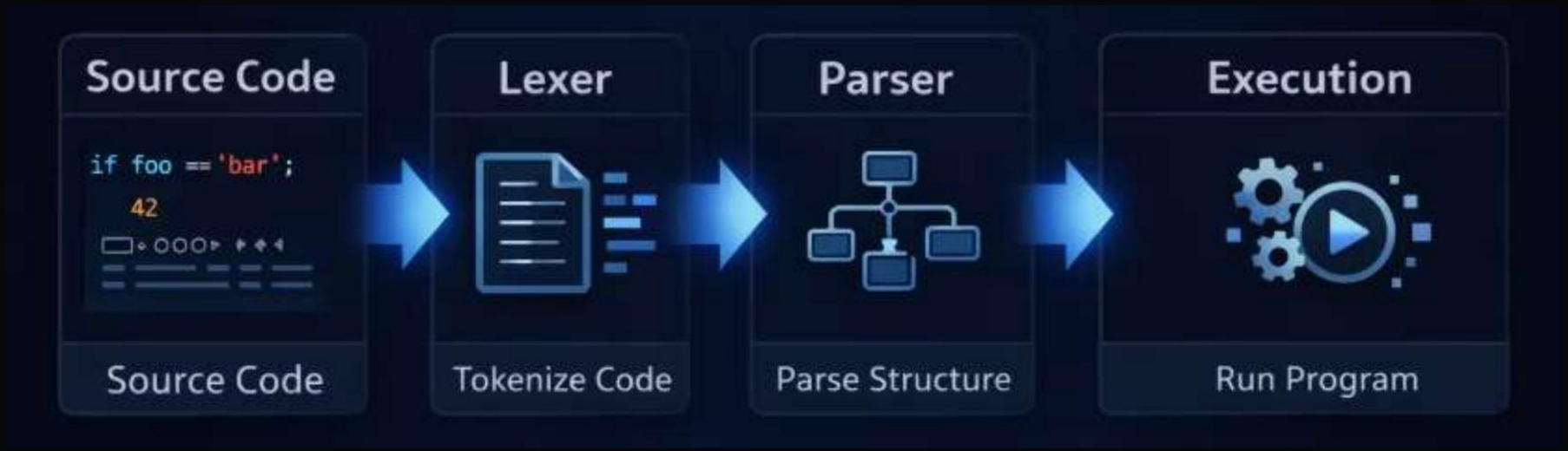
Optimization

Optimizations in loops, dead code elimination and register allocations must be done in such a way that it does not change the expected program behavior, while increasing efficiency of the compiler.

Error Handling & Debugging

Providing meaning error messages so as to make errors understandable to the developers.

Interpreter Pipeline



The initial steps of an interpreter are similar to a compiler. The source code is first tokenized, parsed, and converted into an Abstract Syntax Tree (AST). The AST is then executed directly using depth-first traversal, where expressions are evaluated recursively from leaves to root, without generating machine code.

However, modern interpreters don't just execute code directly from the Abstract Syntax Tree anymore. Instead, they often use compilation techniques like converting the code into bytecode, which is then executed by a virtual machine. Many of them also use Just-In-Time (JIT) compilation to turn frequently used parts of the program into machine code during execution, making the program run faster. In this way, modern language runtimes are really a blend of both interpretation and compilation.

Challenges: With Interpreters

Performance Issue

Execution at runtime costs time. Techniques like bytecode and JIT compilation improve performance, but pre-compiled machine code still offers the highest execution speed.

Dynamic Typing Complexity

Many interpreted languages support dynamic typing, which requires type checking at runtime, increasing execution complexity and slowing down performance.

Limited Optimization

Since execution happens real-time, interpreters can not perform as many deep optimizations as compilers ahead of execution.

Runtime Error Handling

Program may run partially, before crashing completely since errors show up during runtime than before.

Compiler Design

A compiler isn't just a single-step, it is a series of transformations. In the next slides, we will walk through each stage from lexical analysis and parsing to optimization and code generation, and see how high-level code is gradually turned into efficient machine instructions.

We will also look at how these steps are implemented in my compiler.

Compiler Code: ***https://gist.github.com/jyotisko/5555b10fd4337d3f3e7edfbed4f4274d***

1. Lexer (Lexical Analysis)

Purpose:

- Convert raw code into tokens
- Identify keywords, identifiers, numbers and operators of the language
- Handle indentation, spacing and unwanted characters

```
SET x TO 5  →  [Lexer]  →  KEYWORD(SET) IDENTIFIER(x) KEYWORD(TO) NUMBER(5)
```

Popular compilers use a Deterministic Finite Automaton (DFA) to recognize patterns and identify the type of each token. Valid token rules are defined using regular expressions (regex), but since regex can be slow and ambiguous to process directly, tools like Lex or Flex convert these regex into optimized DFA.

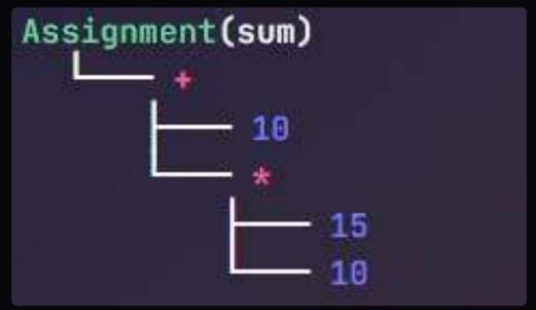
In my compiler, this has been implemented using Regex. The input string is scanned and each matched token is classified and stored.

2. Parser & Abstract Syntax Tree

Purpose:

- Give structure to tokens
- Handle operator precedence and hierarchy

Parser converts a sequence of tokens into a structured representation. Language grammars are checked and output is an AST, which is a tree representation of the program. Each node may represent expression, condition or declarations. The tree is traversed from its depth all the way to top, which takes care of operator precedence.



This is the simplified AST tree for expression **sum = 10 + 15 * 10**

In my compiler, the Parser is validating syntax and constructing the structure simultaneously.

3. Intermediate Representation

AST represents the structure of code, but IR converts it into a simplified, platform-independent form that enables optimization and easy translation to different machine architectures.

For example, a loop to add numbers from 1 to 10 and store them in a variable **sum** would have the following IR code:

```
sum = 0
i = 1

L0:
t1 = i <= 10
if_false t1 goto L1

t2 = sum + i
sum = t2

t3 = i + 1
i = t3

goto L0

L1:
```

IR is generated by traversing AST node by node and producing one or more IR instructions. It uses temporary variables (t1, t2, …) and labels (L0, L1, …).

The IR makes sure the instructions stand clear. Irrespective of the programming language or the system architecture.

4. Optimization

Optimization is one of the most difficult and heavily engineered part of making compilers. The magic that makes a program faster and more efficient happens here. Optimization includes transforming code without changing the program's meaning.

1. Constant Folding

```
int x = 5 + 3;
```

Compiler optimizes the above expression to **x = 8** in the machine code file.

2. Dead Code Elimination

```
int x = 10;
x = 20;
```

Compiler ignores the first initialization to 10 and directly sets x to 20.

3. Loop Optimization and Strength Reduction

```
for (int i = 0; i < 100; ++i) {
    func(i * 1234);
}
```

→ **Optimization**

```
for (int iTimes1234 = 0; iTimes1234 < 100 * 1234; iTimes1234 += 1234) {
    func(iTimes1234);
}
```

Compiler is replacing expensive operations with cheaper ones (e.g., multiplication by constants into shifts or incremental additions in loops). For example, multiplication by 2 is same as left shifting (<<) by 1 bit and division by 2 is same as right shifting (>>) by 1 bit.

4. Common Subexpression Elimination

```
int a = (x * y) + (x * y);
```

→ **Optimization to yield the following IR**

```
t1 = x * y
t2 = t1 + t1
```

Instead of recalculating, the same computation is reused.

In some compilers, LLVM is a compiler infrastructure that performs optimization on intermediate representation (IR) through multiple passes.

Optimization is the art of doing the same work with fewer resources, guided by one rule: never change the meaning of the program. If the compiler is unsure about something, it assumes the worst case and avoids applying optimization.

The final stage includes obtaining machine code from the optimized IR, that can be executed by the CPU.

Disassembler

By using a disassembler, we can turn compiled binaries into assembly and compare how different optimization levels change the final output. This makes it easier to see how compilers reduce unnecessary memory access and make better use of CPU registers to run programs more efficiently.

Assembly: Short Introduction

Assembly is a low-level language that directly represents CPU instructions. Assembly syntax differs for Intel and ARM architectures. We will be focusing on intel syntax, which I find simpler to understand.

Values can be stored in either registers (eax, ebx for example) or memory (denoted by [address]).

```
mov rax, 5      ;# move 5 into rax
mov rbx, rax    ;# move rax into rbx
mov [0x0040100c], rbx   ;# store in memory
```

mov command is used to move data between registers and memory.

;# is used for writing comments.

```
add rax, rbx  ;# add value of rbx into rax
sub rbx, 2    ;# subtract 2 into rbx
```

add and ***sub*** command used for addition and subtraction.

```
_start:
  mov rax, 1     ;# syscall WRITE
  mov rdi, 1     ;# TERMINAL, STD OUT
  lea rsi, [hellostring]
  lea rdx, [hellolen]
  syscall

.data   ;# stores in memory
hellostring: .ascii "Hello world!"
hellolen = . - hellostring
```

Besides is a program to display "Hello world!"

syscall is an instruction used to request services from the operating system, such as input/output or file handling.

lea stands for Load Effective Address and is used to load the address itself.

Disassembler: Walkthrough

Code:

test.c

```c
#include <stdio.h>

int main() {
    int a = 5;
    int b = 10;
    int c = a + b;
    return c;
}
```

Generate the assembly files using GCC, with **–O0** for Unoptimized and **–O2** for Optimized output:

```
gcc.exe -S -O0 test.c -o no_opt.s
gcc.exe -S -O2 test.c -o opt.s
```

Disassembler: Walkthrough

Unoptimized:

no_opt.s

```
.intel_syntax noprefix

.text
.global _main

_main:
    push ebp
    mov ebp, esp
    and esp, -16
    sub esp, 16

    call ___main

    mov DWORD PTR [esp + 12], 5
    mov DWORD PTR [esp + 8], 10

    mov edx, DWORD PTR [esp + 12]
    mov eax, DWORD PTR [esp + 8]
    add eax, edx

    mov DWORD PTR [esp + 4], eax
    mov eax, DWORD PTR [esp + 4]

    leave
    ret
```

Intermediate values are stored in memory and accessed multiple times during computation. This results in more instructions and repeated data movement.

Instead of directly storing the value of 15, it is calculated manually by first storing 5 and 10 in memory and then adding them.

Let us see how it is solved in the optimized version next.

Disassembler: Walkthrough

Optimized:

opt.s

```
.intel_syntax noprefix

.section .text.startup
.global _main

_main:
    push ebp
    mov ebp, esp
    and esp, -16

    call ___main

    mov eax, 15

    leave
    ret
```

The compiler removes intermediate steps and computes the result in advance. The final value is stored directly in a register, reducing memory access and improving efficiency. Registers are faster than memory, as they reside directly on the CPU itself.

Hence, optimization helps reduce unwanted steps and makes the program fast and more efficient.

Summary

- Explored the strengths, weaknesses and internal working pipeline of compilers and interpreters. Realized that building interpreters are relatively easier than building compilers.
- Discussed how the challenges are tackled in modern day compilers and interpreters – using Intermediate Representation, Bytecode, JIT Compilation and Virtual Machines.
- Discussed how optimization is performed by compilers – by eliminating dead code, using registers efficiently, optimization in loops, and converting expensive operations (like multiplications) to simpler operations (like addition) whenever possible.
- Explored the basics of assembly (intel syntax) necessary for understanding the output given by the disassembler.
- Used a disassembler to compare optimized and unoptimized code, observing how optimized code reduces memory usage and number of instructions.
- Languages like C or C++ are preferred in compiler development. Built a small compiler in JavaScript and documented the full pipeline. Chose JavaScript to focus more on understanding concepts and implementation.

Bibliography

Below is a list of all the resources I used throughout the task:

- **KOSS Task Page – Compilers** – Understanding the task and using the provided resources
- **Build Your Own X (X=Programming Language)**
- **A Compiler Writing Journey**
- **Assembly For Beginners Playlist** – Learning the basics of assembly
- **Beautiful Code Snippets** – Attaching beautiful snippets of code
- **ChatGPT** – Clarification and understanding

Thank You